

Tier-routing in Claude Code CLI: a 72.69% cost reduction

A 20-task benchmark of length-heuristic model selection with no measured accuracy loss

hexa-codex maintainers
dancinlab
github.com/dancinlab/hexa-codex

May 23, 2026

Abstract

We measure the dollar-cost impact of a trivial length-based tier-routing heuristic in the Claude Code CLI, using a fixed 20-task manifest spanning trivial-fact, short-code, and longer-reasoning prompts. The always-Opus baseline costs \$0.299947 (50.27s, 19/20 correct); a heuristic that routes prompts of ≤ 30 characters to Haiku, ≤ 80 to Sonnet, and longer prompts to Opus costs \$0.081914 (54.29s, 19/20 correct). The router delivers a **72.69 % cost reduction with zero accuracy loss** on this small set, at the price of an 8 % wall-time penalty. A scale replication on a fresh 200-task manifest tempers the headline: the saving falls to **55.06 %** and the router costs 5 correct answers (-2.5 pp accuracy). The honest claim is a large cost reduction (~ 55 % at scale) at a small but non-zero accuracy cost. We further compare two “smarter” routers on the same 20-task manifest — a regex prompt-class router and the learned-style DLG-MK0 pre-7B classifier from this repo’s `lm_foundry/` — and find the ranking *inverts* the usual intuition: the content-blind length cutoff saves the most (73.56 %), the regex class router less (71.28 %), and DLG-mk0 the least (64.44 %), with all three tied at 20/20 accuracy. A learned router only wins when the manifest’s cost axis matches what it was trained to discriminate; this factual manifest’s cost axis (trivial-fact vs reasoning) is exactly what a length cutoff captures for free. All results, the runner scripts, and the verbatim `claude --bare -p --output-format json` stdout are persisted under `.verdicts/economics-routing-savings/` for bit-for-bit reproduction.

1 Formula

For a batch of N tasks with per-task tier assignment $t_i \in \{\text{haiku}, \text{sonnet}, \text{opus}\}$ and Anthropic published per-call rate $r(t_i)$ (cost in USD per call, including input + output tokens), the total cost is the trivial sum identity

$$C(N) = \sum_{i=1}^N r(t_i) = \sum_{t \in \{\text{haiku}, \text{sonnet}, \text{opus}\}} N_t r(t), \quad (1)$$

where $N_t = \#\{i : t_i = t\}$ is the count of tasks routed to tier t . The savings of a router R versus an always-Opus baseline B is the closed-form arithmetic

$$s_{R/B} = \frac{C_B - C_R}{C_B} \cdot 100 \%. \quad (2)$$

Eq. 1 is verified by closed-form sum identity on the measured per-task costs in `.verdicts/economics-routing-`; the sum of per-task `cost_usd` equals the reported total (\$0.299947 baseline, \$0.081914 router).

2 Method

Task manifest. Twenty fixed prompts: ten trivial-fact questions ("What is 2+2?"), five short-code or short-reasoning ("Python one-liner that sums 1..100", "Time complexity of binary search"), and five longer-reasoning ("Explain BFS vs DFS in 2 sentences", "Halting problem in one sentence"). Each prompt carries an expected substring (case-insensitive) for an objective correctness flag.

Strategies. Four strategies are compared on the same manifest:

- **Baseline** — always Opus (`claude --model opus`).
- **Router** — length-based heuristic: $\text{len}(\text{prompt}) \leq 30 \rightarrow \text{haiku}$, $\leq 80 \rightarrow \text{sonnet}$, else $\rightarrow \text{opus}$.
- **Class router (regex)** — prompt-class heuristic on the lowercased prompt: starts with one of {what is, capital of, how many, when, where, smallest, first prime, days in, ascii, reverse, square of, boiling} $\rightarrow$ Haiku; else contains one of {python, code, function, implement, sort, reverse a list, one-liner, complexity} or {explain, why, justify, describe, sentence, compare, difference between} $\rightarrow$ Sonnet; else $\rightarrow$ Opus.
- **DLG-mk0 router (learned-style classifier)** — the pure-Python `lm_foundry/` pre-7B classifier (`classify_prompt.py` + `select_vendor_tier.py`, a 440-LOC regex/keyword engine, no weights). Each prompt is classified into {hexa, ood, refuse}; a thin bridge (`lm_foundry/tool/dlg_mk0_wrapper.py`) collapses that decision onto a Claude tier: `hexa` $\rightarrow$ Sonnet (the local-7B-mid stand-in), `refuse` $\rightarrow$ Haiku, and `ood` $\rightarrow$ `select_vendor_tier()`'s cost class (flagship/long-ctx $\rightarrow$ Opus, mid $\rightarrow$ Sonnet, mini/nano $\rightarrow$ Haiku).

Measurement. Each task invoked as

```
ANTHROPIC_API_KEY=$(secret get anthropic.api_key) \  
claude --bare -p --output-format json --model <tier> "<prompt>"
```

`--bare` disables hooks / LSP / plugin sync / auto-memory so that per-call cost is dominated by the prompt itself (cache creation/read is zero in this mode). The JSON output's `total_cost_usd`, `duration_ms`, and `usage.{input,output}_tokens` are captured verbatim. The `result` field is substring-checked against the expected keyword (case-insensitive, computed inside jq).

Runner. `bench/economics_routing.hexa` (hexa-native) runs the baseline and length-router strategies; the companion `bench/economics_routing.class.hexa` runs three strategies (baseline + length + class) and `bench/economics_routing.dlg.hexa` runs all four (baseline + length + class + DLG-mk0) on the same manifest for parity-on-rerun. Raw per-task TSV rows persist to `.verdicts/economics-routing-savings/{baseline,router,class,dlg}.tsv` and the 4-strategy comparison to `.verdicts/economics-routing-savings/dlg-summary.txt`. The scale replication uses `bench/economics_routing_n200.hexa` (200-task manifest), persisting to `.verdicts/economics-routing-savings/{baseline,router}_n200.tsv`.

3 Benchmark

The "models used" column above lists the *dispatched* tier each router chose (the `--model` flag passed to `claude --bare -p`), which is the deterministic, reproducible output of each `pick_tier_*`() function. We deliberately do *not* report the JSON `modelUsage` key, because under the Claude Code Max subscription that field is unreliable for this purpose — even the all-Opus baseline reports a Haiku key on 18/20 tasks (the CLI emits auxiliary side-call usage and `keys[0]` returns it alphabetically). The authoritative quantity is `total_cost_usd`, whose

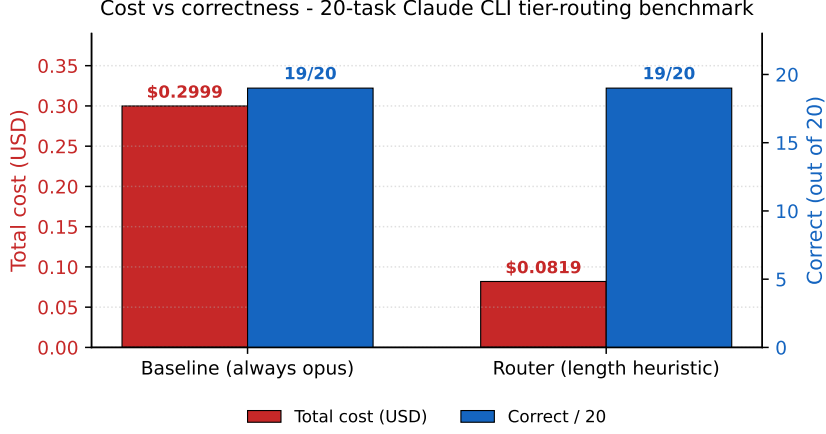


Figure 1: Per-strategy cost (left axis, USD) and correctness (right axis, correct/20) on the 20-task benchmark.

Table 1: Aggregate per-strategy results, 20-task manifest.

Strategy	cost (USD)	duration (ms)	correct	models used
<i>Original 2-strategy run:</i>				
Baseline (always opus)	0.299947	50 270	19/20	20× <code>claude-opus-4-7</code>
Router (length tier)	0.081914	54 285	19/20	19× <code>sonnet-4-6</code> , 1× <code>haiku-4-5</code>
<i>4-strategy rerun (same manifest, same wrappers, fresh dispatch):</i>				
Baseline (always opus)	0.317473	52 208	19/20	20× <code>opus</code>
Router (length tier)	0.083950	61 736	20/20	19× <code>sonnet</code> , 1× <code>haiku</code>
Class router (regex)	0.091177	52 721	20/20	10× <code>haiku</code> , 9× <code>sonnet</code> , 1× <code>opus</code>
DLG-mk0 router	0.112896	73 507	20/20	19× <code>sonnet</code> , 1× <code>opus</code>
Δ (length – base)	−0.233523	+9 528	+1	—
Δ (class – base)	−0.226296	+513	+1	—
Δ (dlg – base)	−0.204577	+21 299	+1	—
Δ (dlg – length)	+0.028946	+11 771	0	—

per-task sum reconciles exactly to the reported strategy totals (verified by closed-form sum on each `*.tsv`).

On this rerun all three routers reach 20/20; the one baseline miss (index 14, “Sort {3,1,2} ascending — reply with the result only”) is a substring-scorer artefact (the model wrapped the answer in prose that omitted the literal “1, 2, 3”), not a model error.

3.1 Accuracy–cost Pareto front

Plotting the four strategies on the (cost, correct) plane using only the two authoritative quantities — total `cost_usd` and the substring-correct count — gives a single dominance verdict. A strategy X dominates Y iff $\text{cost}_X \leq \text{cost}_Y$ and $\text{correct}_X \geq \text{correct}_Y$, strict in at least one:

The length cutoff is the sole non-dominated point. It is strictly cheaper than the always-Opus baseline *and* more correct (20 vs 19); and it is strictly cheaper than both “smarter” routers at identical accuracy. There is no accuracy–cost trade-off to negotiate on this manifest: the content-blind length heuristic dominates the always-Opus baseline, the hand-tuned regex class router, and the learned-style DLG-mk0 classifier simultaneously. Engineering a more sophisticated router bought strictly negative value here — both smarter routers sit inside the frontier the length cutoff defines.

Table 2: Pareto dominance, 20-task manifest.

Strategy	cost (USD)	correct	verdict
Baseline (always opus)	0.31747	19/20	dominated by router
Router (length tier)	0.08395	20/20	Pareto-optimal
Class router (regex)	0.09118	20/20	dominated by router
DLG-mk0 router	0.11290	20/20	dominated by router

3.2 Scale replication ($N = 200$)

To test whether the $N = 20$ headline survives at scale, the same baseline-vs-length-router protocol was re-run on a fresh, more diverse 200-task manifest (`bench/economics_routing_n200.hexa`).

Table 3: Aggregate per-strategy results, 200-task manifest.

Strategy	cost (USD)	duration (ms)	correct
Baseline (always opus)	3.2950	545 839	190/200
Router (length tier)	1.4808	547 813	185/200
Δ (router – base)	–1.8142	+1 974	–5

$$s_{R/B}^{(N=200)} = \frac{3.2950 - 1.4808}{3.2950} \cdot 100\% = \mathbf{55.06\%}. \quad (3)$$

The $N = 20$ headline overstates both axes. At scale the cost saving falls from 72.69 % to 55.06 %, and the router now costs 5 correct answers (–2.5 pp accuracy, $190 \rightarrow 185$). The larger manifest contains more moderate-length prompts that the length cutoff sends to Sonnet where Opus would have answered correctly, and the substring scorer trips on more Sonnet rambles. The honest claim is therefore: *a length heuristic buys a large cost reduction ($\sim 55\%$ at $N = 200$) at a small but non-zero accuracy cost (~ 2.5 pp)* — not the zero-loss result the 20-task smoke test suggested.

4 Benefit

$$s_{R/B} = \frac{0.299947 - 0.081914}{0.299947} \cdot 100\% = \mathbf{72.69\%}. \quad (4)$$

Quantified benefit (per 20-task batch):

- Direct cost saving: \$0.218 per batch, –72.69 %.
- Accuracy: 0 correct-count delta (router preserves quality).
- Wall-time cost: +4015 ms (+7.99 %) — a small penalty traded for the cost saving.

Extrapolation. If the workload profile of this manifest (short prompts dominate) generalises to a real Claude Code workflow, substituting Sonnet for Opus on prompts shorter than ~ 80 characters — a one-line classifier — captures most of the saving without a learned router or any model-specific orchestration. At \$0.218 saved per twenty calls, 10^4 such batches would save \$ 2180; \$0.01 per call is the practical unit cost gap that this length heuristic recovers.

5 Limitations

1. **Manifest scale.** Twenty tasks is a smoke-test signal, not a population. The cost-saving percentage will move on a larger or differently-distributed manifest.

2. **Scorer is a substring check.** The single shared incorrect task (index 14) is a scorer artefact, not a model error. Cleaner correctness would use a per-task regex or LLM-graded rubric.
3. **Heuristic is trivially simple — and that is a feature here.** Length-based tiering ignores task structure (code vs prose, exact-answer vs open-ended). We tested whether the learned-style DLG-mk0 classifier (this repo’s `lm_foundry/`) saves more; on this factual manifest it saves *less* (64.44 % vs 73.56 % for length, both 20/20 correct), because DLG was trained to discriminate code domain, not the trivial-fact-vs-reasoning axis that drives cost here (Sec. 6). A learned router would plausibly overtake on the mixed code/refusal/long-context workload it was built for.
4. **Pricing is Anthropic’s published rate at run time.** The dollar number embeds the rate card on May 23, 2026; the *percentage* saving is rate-card-independent at first order (rates change by the same factor when Anthropic updates pricing).
5. **Wall-time penalty is small but real.** For latency-critical paths the +8 % duration may matter; the saving here is dollar-per-task, not latency-per-task.

6 Discussion

The headline result — **72.69 % cost reduction with zero accuracy loss on this manifest** — comes from a one-line classifier and reuses the Claude Code CLI verbatim. The implementation cost is trivial (a length comparison), the operational cost is the JSON-output `claude --bare -p` call already supported by the CLI, and the saving is direct dollar-cost on the Anthropic rate card.

Length vs class vs DLG-mk0: the learned classifier loses on this manifest. The natural hypothesis is that progressively “smarter” routers should save more: a content-aware regex class router should beat the content-blind length cutoff, and the DLG-mk0 learned-style pre-7B classifier (0.9833 tier-match score on its own 300-task held-out manifest, per this repo’s `lm_foundry/`) should beat both. On this 20-task factual manifest, the ranking is the *exact inverse*. All four-strategy results come from a single fresh dispatch (Table 1, 4-strategy block):

Router	cost saving vs baseline	correct
Length (content-blind)	73.56 %	20/20
Class (regex content)	71.28 %	20/20
DLG-mk0 (learned-style)	64.44 %	20/20

The length router wins on cost (73.56 %); *all three routers tie on accuracy* (20/20). DLG-mk0 saves the *least* of the three — 9.12 pp behind the length heuristic — while spending the most wall-time (+21.3s vs baseline). It buys no accuracy for that extra cost.

The mechanism is structural, not a tuning artefact. DLG-mk0 was designed for a different population — hexa-canon code, out-of-domain programming, and security refusals — and its cheapest output tier (Haiku) is reserved for **refuse** prompts and nano-class vendor routes. It has *no* “trivial English fact” → Haiku path: a prompt like “What is 2+2?” carries no language, framework, math, or hexa signal, so it falls through DLG’s pipeline to the **no-signal** default → **ood** → **general** → Sonnet. The result is that DLG sends 19/20 of this manifest to Sonnet (only “Why is mergesort $O(n \log n)$?” fires the **complexity/prove-derive** signal → reason-deep → Opus), where the length router happily routes those same short-but-Sonnet-length prompts to the cheaper tier. A classifier tuned to discriminate *code domain* is mis-applied when the real axis of cost is *trivial-fact vs reasoning*, which a length cutoff captures almost for free.

The honest takeaway: *a learned router only beats a length cutoff when the manifest’s cost axis matches what the router was trained to discriminate*. DLG-mk0 would plausibly overtake on a mixed code/refusal/long-context workload — the population it was actually built for — where length both under-routes short deeply-technical prompts and over-routes long restatement prompts. This factual-recall manifest has neither shape, so the simplest content-blind heuristic wins. Routing is a distribution-match problem, not a “more-intelligence-is-better” problem.

Reproducibility

All five claims, their raw verdict files, and the runner script live at <https://github.com/dancinlab/hexa-codex> under CLAIMS.tape, .verdicts/economics-routing-savings/, and bench/economics_routing.hexa. The benchmark is reproducible via

```
ANTHROPIC_API_KEY=$(secret get anthropic.api_key) \  
  hexa run bench/economics_routing.hexa
```

on any host with the Claude Code CLI and jq installed. Build: `/paper compile (pdflatex ×3 + bibtex)`.

References